UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA

Facultad de Ingeniería de Informática y Sistemas

Escuela Profesional de Ingeniería en Ciberseguridad

PROYECTO DE INVESTIGACIÓN:

CHECKLIST DE HARDENING: APLICACIÓN WEB PARA REVISAR LA SEGURIDAD DE UN EQUIPO O SERVICIO

Alumnos :

Ponce Ramirez, Arnold Cesar

Lazo Gonzales, Sayury Shandal

Equipo 7

Docente : Jara Linares, Jorge Luis

Curso : Programación II (IC010404)

Ciclo : 2026-II

Tingo María – Perú

Checklist de hardening: aplicación web para revisar la seguridad de un equipo o servicio

Antes de poner un equipo o servicio a funcionar hay que hacer ajustes que cierren las puertas que usaría un atacante. A esto se le dice hardening (endurecimiento en español). Casi siempre se hace de memoria y no queda registro de lo revisado. Este informe presenta Checklist de Hardening una app web que guía la revisión con listas de controles y guarda la evidencia. La hicimos para Programación II con solo HTML CSS y JavaScript sin frameworks ni librerías.

Índice General

Descripción del proyecto 4
Decisiones de diseño 5
Estructura y código 5
Diseño responsive 5
Usabilidad y accesibilidad 5
Pruebas manuales 6
Seguridad 7
Reflexión sobre frameworks 8
Herramientas de apoyo 9
Conclusión 10
Referencias 11

Descripción del proyecto

La app es para administradores de sistemas y auditores. Tiene 4 páginas con el mismo menú: Inicio, Auditorías, Checklist y Reporte. En Auditorías se registra una revisión con nombre del activo, responsable, fecha y tipo de lista (sistema operativo, servidor web o red Wi-Fi). Cada tipo trae 8 controles ej “HTTPS con certificado válido” o “Firewall del sistema activo”. En Checklist se marca cada control como pendiente, aplicado o no aplica y se escribe una nota de evidencia. En Reporte se ve el avance, el % por categoría y los pendientes de prioridad alta y se puede imprimir.

Definimos 2 reglas propias. 1ra el avance no cuenta los controles “no aplica”: si 3 de 8 están aplicados y 1 es “no aplica” el avance es 43 % (3 de 7). 2da una auditoría con controles de prioridad alta pendientes solo se cierra si se escribe una justificación. También se puede duplicar una auditoría, cargar 10 de ejemplo, buscar, filtrar por tipo y estado, ordenar, ver resumen con contadores y exportar los datos en JSON. Todos los datos son ficticios.

Decisiones de diseño

Estructura y código

Usamos etiquetas semánticas de HTML5 (`header`, `nav`, `main`, `section` y `footer`) porque ayudan a los lectores de pantalla a entender la página (W3C, 2023). En el código separamos 2 tareas: `datos.js` guarda lee y calcula y `auditoria.js`, `checklist.js` y `reporte.js` manejan lo que se ve en pantalla. Los datos se guardan en el navegador con `localStorage` que solo admite texto asi que los pasamos a JSON (Mozilla, s. f.). El id de la auditoría q se está revisando va en una 2da clave para que Checklist y Reporte sepan cuál mostrar. Para trabajar en equipo usamos Git y GitHub con una rama por etapa, pull requests y etiquetas de versión (Chacon y Straub, 2014) y publicamos el sitio en GitHub Pages (GitHub, s. f.)

Diseño responsive

Diseñamos primero para pantallas chicas y luego ajustamos para tablets (desde 601 px) y escritorio (desde 961 px) con media queries. Los colores y espacios están en variables CSS y todos los elementos usan `box-sizing: border-box` para que nada se salga de su tarjeta. El menú usa Flexbox y la estructura general usa Grid para que el pie quede siempre abajo. También agregamos reglas de impresión que ocultan menú, botones y pie. Una limitación: en celulares los enlaces del menú se acomodan en filas pero no hay botón desplegable. Con más tiempo lo agregaríamos con el atributo `aria-expanded`.

Usabilidad y accesibilidad

Cada campo tiene su `label`. Cuando hay error el mensaje sale junto al campo empieza con la palabra “Error:” y se conecta al campo con `aria-describedby`, `aria-invalid` y `role="alert"`. No usamos solo el rojo porque hay personas q no lo perciben bien (W3C, 2023). La interfaz muestra el foco del teclado, controles deshabilitados, mensaje de lista vacía y mensajes de éxito. Aplicamos varias heurísticas de Nielsen (1994): la barra de avance muestra el estado del sistema, el cierre se bloquea si falta la justificación (prevención de errores) y están los botones Cancelar y Reabrir auditoría (control del usuario). Eliminar y cerrar piden confirmación.

Pruebas manuales

Probamos la app a mano en el navegador con datos ficticios siguiendo casos con precondición, pasos, resultado esperado y resultado obtenido. La Tabla 1 resume los 10 casos ejecutados y todos aprobaron. El detalle está en `docs/pruebas-y-seguridad.md` del repositorio.

Tabla 1

Casos de prueba manuales ejecutados

ID	Qué se probó	Resultado
CP-01	El avance excluye “no aplica” (prueba de aceptación: 43 %)	Aprobado
CP-02	Se bloquea el cierre con prioridad alta pendiente y sin justificación	Aprobado
CP-03	Cierre válido de la auditoría con justificación	Aprobado
CP-04	Duplicar una auditoría existente	Aprobado
CP-05	Validación del formulario vacío con mensajes accesibles	Aprobado
CP-06	Texto con código (XSS) se muestra como texto, sin ejecutarse	Aprobado
CP-07	Eliminar con confirmación	Aprobado
CP-08	Reporte por categoría y vista de impresión	Aprobado
CP-09	Cargar 10 auditorías de ejemplo; se mantienen al recargar	Aprobado
CP-10	Exportar JSON: se descarga el archivo; con lista vacía avisa que no hay datos	Aprobado

Nota. Elaboración propia. Todos los datos usados son ficticios.

Seguridad

Revisamos 6 controles de seguridad. 1ro no hay claves contraseñas ni datos reales: los nombres de equipos y responsables son inventados. 2do todo texto que escribe el usuario se muestra con `textContent` y nunca con `innerHTML` y no usamos `eval`. Esto evita el ataque XSS donde alguien escribe código en un campo para q se ejecute en la página (OWASP Foundation, 2025). La prueba CP-06 lo comprueba. 3ro los campos se validan con JavaScript pero eso no reemplaza la validación de un servidor porque el usuario puede modificar el código de su navegador. 4to los únicos enlaces externos son los perfiles del pie y llevan `rel="noopener noreferrer"`; la app no acepta enlaces escritos por el usuario. 5to `localStorage` guarda solo datos no sensibles: no está cifrado cualquier script de la página puede leerlo y tiene un límite cercano a 5 MB (Mozilla, s. f.). 6to cada página define una política de seguridad de contenido (CSP) en una etiqueta `meta` que solo permite recursos propios y no usamos scripts de terceros.

Reflexión sobre frameworks

En nuestro proyecto cada vez que cambia un dato hay q vaciar el contenedor y crear los elementos otra vez a mano. Un framework resuelve eso: el programador describe cómo debe verse la pantalla según el estado de los datos y el framework actualiza la página solo. Su base son los componentes, piezas reutilizables q juntan estructura, estilo y lógica. En nuestra app serían componentes el menú de navegación (`MenuNavegacion`), la barra de avance (`BarraAvance`), la tarjeta de cada auditoría (`TarjetaAuditoria`) y cada control del checklist (`ItemControl`). React es una biblioteca para construir interfaces que usa JSX, o sea HTML escrito dentro de JavaScript (Meta Platforms, Inc., s. f.). Vue es un framework progresivo que separa plantilla, estilo y código en un solo archivo por componente (You, s. f.). Elegimos JavaScript puro porque nos obligó a entender cómo funciona el navegador. Si la app creciera los 4 archivos `.js` podrían pasar a un árbol de componentes con estado compartido

Herramientas de apoyo

Usamos 4 herramientas de apoyo. Claude (Anthropic, 2026) y Gemini (Google, 2026) 2 asistentes de IA nos ayudaron a explicar conceptos, revisar el código, encontrar errores (ej funciones repetidas) y mejorar la redacción de la documentación. La plataforma Academia HolaMundo (HolaMundo, s. f.) y videos de YouTube (Google, s. f.) nos sirvieron para aprender y repasar HTML, CSS y JavaScript. Las herramientas fueron una ayuda: el equipo probó la app, revisó los resultados y puede explicar cómo funciona cada parte del código

Conclusión

Logramos una app q ordena la revisión de hardening, calcula el avance con una regla clara y deja evidencia. Aprendimos a separar los datos de la pantalla, a pensar en accesibilidad desde el inicio y a proteger una página web con prácticas básicas. Como mejoras futuras quedan el botón desplegable del menú, un servidor con validación propia y trabajar en equipo con datos compartidos

Referencias

Anthropic. (2026). *Claude* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book

GitHub. (s. f.). *GitHub Pages documentation*. https://docs.github.com/pages

Google. (s. f.). *YouTube* [Plataforma de video]. https://www.youtube.com

Google. (2026). *Gemini* [Modelo de lenguaje de gran tamaño]. https://gemini.google.com

HolaMundo. (s. f.). *Academia HolaMundo* [Plataforma de cursos en línea]. https://academia.holamundo.io

Meta Platforms, Inc. (s. f.). *React documentation: Describing the UI*. https://react.dev/

Mozilla. (s. f.). *Web Storage API - MDN Web Docs*. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '94)* (pp. 152-158). ACM.

OWASP Foundation. (2025). *OWASP Top 10:2025*. https://owasp.org/Top10/2025/

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

You, E. (s. f.). *Vue.js guide: Essentials and components in-depth*. https://vuejs.org/guide/